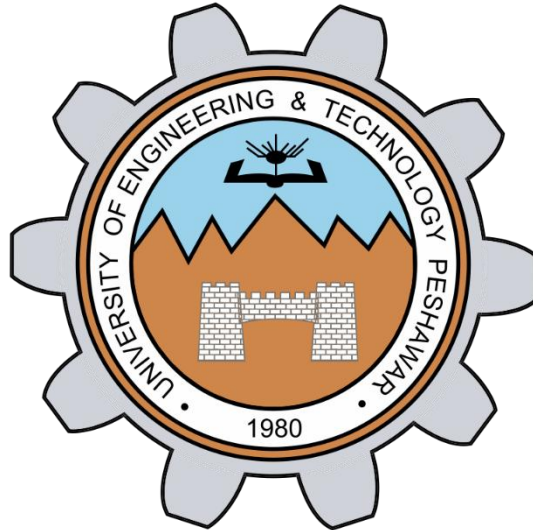


UNIVERSITY OF ENGINEERING AND TECHNOLOGY,
PESHAWAR PAKISTAN
Main Campus



CS-224 Web Technologies
Assignment No. 02

Submitted By

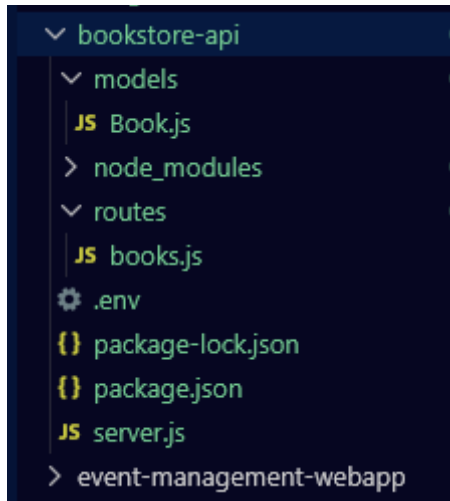
Name:	Hisham khan
Registration No.	24pwbc1402
Semester:	BS CS 4th
Section:	E
Submission Date:	

Submitted To
Sir Mohammad

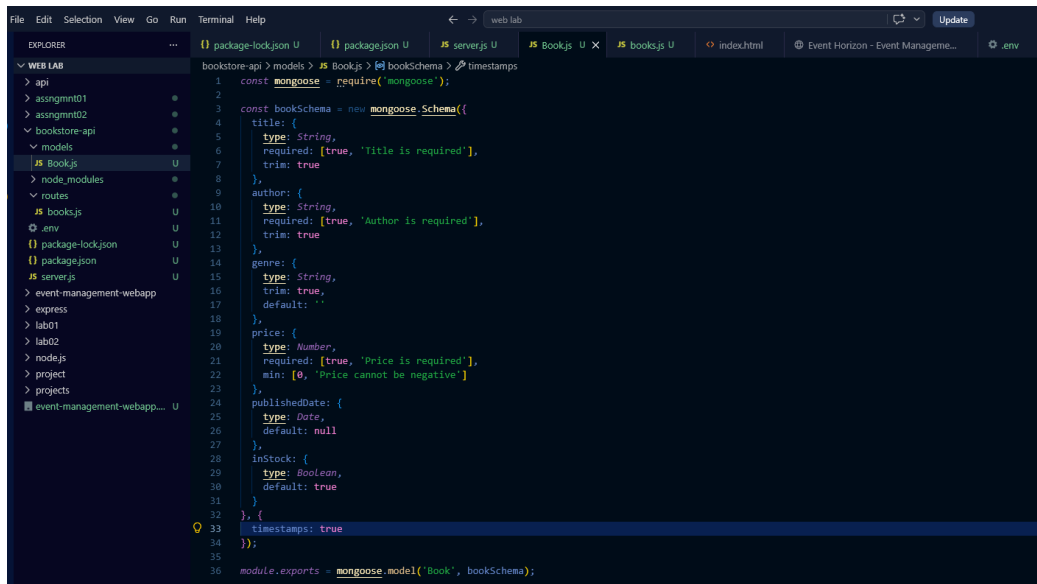
DEPARTMENT OF COMPUTER SCIENCE & IT
UNIVERSITY OF ENGINEERING AND TECHNOLOGY, PESHAWAR, PAKISTAN

CODE

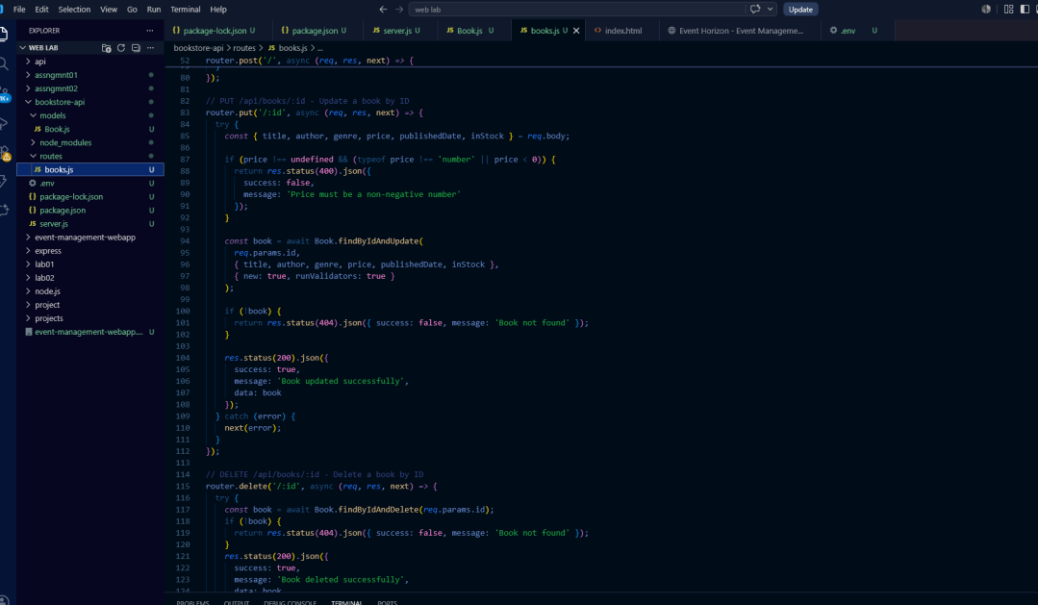
Structure



Models/books.js



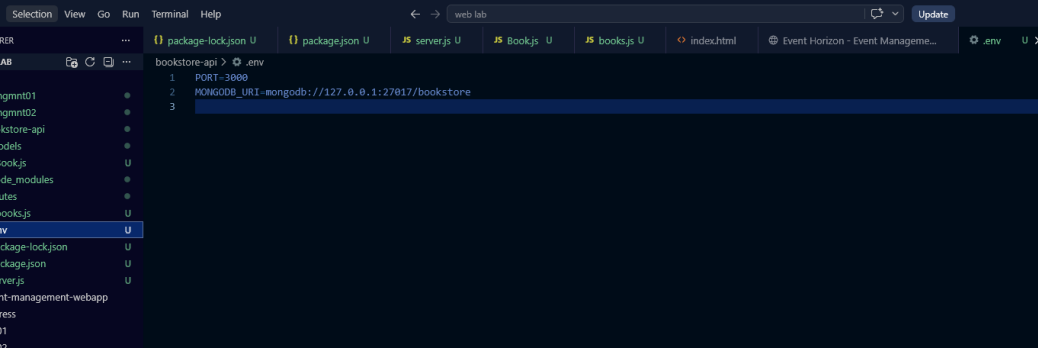
Route/book.js



The screenshot shows a web lab interface with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'api', 'assignment01', 'assignment02', 'bookstore-api', 'models', 'routes', and 'books.js'. The 'books.js' file is selected. The code editor displays the following JavaScript code:

```
17 router.post('/', async (req, res, next) => {
18   // PUT /api/books/:id - Update a book by ID
19   router.put('/:id', async (req, res, next) => {
20     try {
21       const { title, author, genre, price, publishedDate, inStock } = req.body;
22       if (price != undefined && (typeof price != 'number' || price < 0)) {
23         return res.status(400).json({
24           success: false,
25           message: 'Price must be a non-negative number'
26         });
27       }
28       const book = await Book.findByIdAndUpdate(
29         req.params.id,
30         { title, author, genre, price, publishedDate, inStock },
31         { new: true, runValidators: true }
32       );
33       if (!book) {
34         return res.status(404).json({ success: false, message: 'Book not found' });
35       }
36       res.status(200).json({
37         success: true,
38         message: 'Book updated successfully',
39         data: book
40       });
41     } catch (error) {
42       next(error);
43     }
44   });
45   // DELETE /api/books/:id - Delete a book by ID
46   router.delete('/:id', async (req, res, next) => {
47     try {
48       const book = await Book.findByIdAndDelete(req.params.id);
49       if (!book) {
50         return res.status(404).json({ success: false, message: 'Book not found' });
51       }
52       res.status(200).json({
53         success: true,
54         message: 'Book deleted successfully',
55         data: book
56       });
57     } catch (error) {
58       next(error);
59     }
60   });
61 }
```

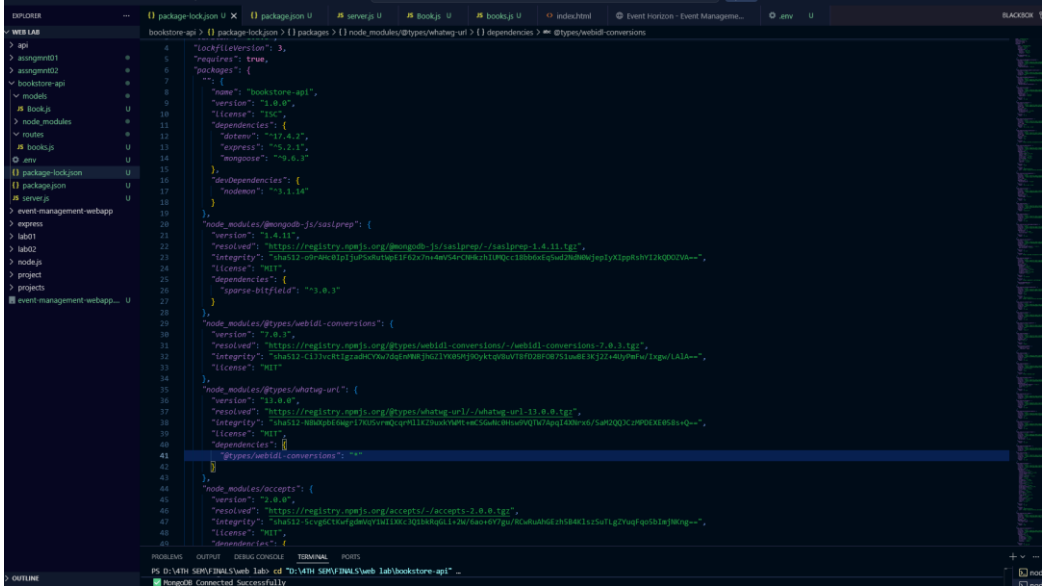
Env



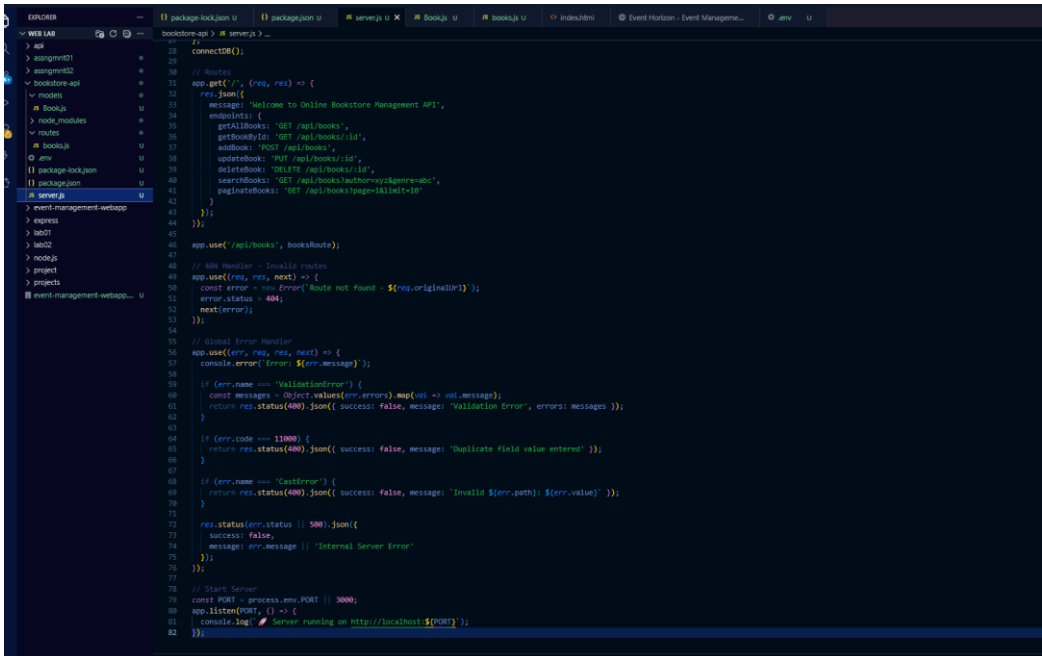
The screenshot shows a web lab interface with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'api', 'assignment01', 'assignment02', 'bookstore-api', 'models', 'routes', and 'books.js'. The '.env' file is selected. The code editor displays the following environment variables:

```
1 PORT=3000
2 MONGODB_URI=mongodb://127.0.0.1:27017/bookstore
3
```

Package_lock.js



Server.js



1. GET Single Book by ID

The screenshot displays a REST client interface with a request and response view. The request is a GET method to the endpoint `http://localhost:3000/api/books/6a2195bcbd0bb7d6ba5a1eab`. The response is a 200 OK status with a response time of 19 ms and a body size of 523 B. The response body is a JSON object representing a book.

Request:

- Method: GET
- URL: `http://localhost:3000/api/books/6a2195bcbd0bb7d6ba5a1eab`

Response:

200 OK • 19 ms • 523 B

```
{
  "success": true,
  "data": {
    "_id": "6a2195bcbd0bb7d6ba5a1eab",
    "title": "To Kill a Mockingbird",
    "author": "Harper Lee",
    "genre": "Classic",
    "price": 14,
    "publishedDate": "1960-07-11T00:00:00.000Z",
    "inStock": false,
    "createdAt": "2026-06-04T15:11:56.536Z",
    "updatedAt": "2026-06-04T15:11:56.536Z",
    "__v": 0
  }
}
```

2. UPDATE a Book (PUT)

The screenshot shows the Postman application interface. On the left, the 'COLLECTIONS' sidebar is visible, showing a collection named 'My Collection' with two items: 'GET Get data' and 'POST Post data'. The main workspace displays a PUT request to the URL 'http://localhost:3000/api/books/6a2195bcbd0bb7d6ba5a1eab'. The request body is a JSON object with 'price' set to 20.00 and 'inStock' set to true. Below the request, the response is shown as a JSON object with a success message and the updated book data.

Overview: [CONFLI] GET Get da POST Post My Coll My Coll Post da + No environment

My Collection > Get data

PUT http://localhost:3000/api/books/6a2195bcbd0bb7d6ba5a1eab Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookie

raw JSON Schema Beautify

```
1 {
2   "price": 20.00,
3   "inStock": true
4 }
```

Body 200 OK • 133 ms • 560 B Save Response

{ JSON Preview Visualize

```
1 {
2   "success": true,
3   "message": "Book updated successfully",
4   "data": {
5     "_id": "6a2195bcbd0bb7d6ba5a1eab",
6     "title": "To Kill a Mockingbird",
7     "author": "Harper Lee",
8     "genre": "Classic",
9     "price": 20,
10    "publishedDate": "1960-07-11T00:00:00.000Z",
11    "inStock": true,
12    "createdAt": "2026-06-04T15:11:56.536Z",
13    "updatedAt": "2026-06-04T23:27:45.983Z",
14    "__v": 0
15  }
16 }
```

> ENVIRONMENTS

3. DELETE a Book (DELETE)

The screenshot displays a REST client interface with a sidebar on the left containing 'COLLECTIONS' (Collection, Get data, Post data, Post data) and 'COMMENTS' (3). The main area shows a 'DELETE' request to the URL 'http://localhost:3000/api/books/6a2195bcd0bb7d6ba5a1eab'. The 'Body' tab is selected, showing a JSON object:

```
{ 1: { 2:   "price": 20.00, 3:   "inStock": true 4: }}
```

. Below the request, the response is shown with a status of '200 OK', a duration of '24 ms', and a size of '560 B'. The response body is a JSON object:

```
{ 1: { 2:   "success": true, 3:   "message": "Book deleted successfully", 4:   "data": { 5:     "_id": "6a2195bcd0bb7d6ba5a1eab", 6:     "title": "To Kill a Mockingbird", 7:     "author": "Harper Lee", 8:     "genre": "Classic", 9:     "price": 20, 10:    "publishedDate": "1960-07-11T00:00:00.000Z", 11:    "inStock": true, 12:    "createdAt": "2026-06-04T15:11:56.536Z", 13:    "updatedAt": "2026-06-04T23:27:45.983Z", 14:    "__v": 0 15:   } 16: }
```

4. SEARCH by Author

The screenshot displays the Postman application interface. On the left, the 'COLLECTIONS' sidebar shows 'My Collection' with two items: 'GET Get data' (selected) and 'POST Post data'. The main workspace is titled 'My Collection > Get data'. The request is a GET method to the URL 'http://localhost:3000/api/books?author=Fitzgerald'. The response is a 200 OK status with a response time of 170 ms and a body size of 851 B. The response body is shown in JSON format, displaying a successful search result for books by the author 'F. Scott Fitzgerald'.

Request:

```
GET http://localhost:3000/api/books?author=Fitzgerald
```

Response (JSON):

```
{
  "success": true,
  "count": 2,
  "total": 2,
  "totalPages": 1,
  "currentPage": 1,
  "data": [
    {
      "_id": "6a2195a6bd0bb7d6ba5a1eaa",
      "title": "The Great Gatsby",
      "author": "F. Scott Fitzgerald",
      "genre": "Classic",
      "price": 15.99,
      "publishedDate": "1925-04-10T00:00:00.000Z",
      "inStock": true,
      "createdAt": "2026-06-04T15:11:34.397Z",
      "updatedAt": "2026-06-04T15:11:34.397Z",
      "__v": 0
    },
    {
      "_id": "6a2194e0bd0bb7d6ba5a1ea4",
      "title": "The Great Gatsby",
      "author": "F. Scott Fitzgerald",
      "genre": "Classic",
      "price": 15.99,
      "publishedDate": "1925-04-10T00:00:00.000Z",
      "inStock": true,
      "createdAt": "2026-06-04T15:11:34.397Z",
      "updatedAt": "2026-06-04T15:11:34.397Z",
      "__v": 0
    }
  ]
}
```


5. SEARCH by Genre

The screenshot shows the Postman application interface. At the top, the browser address bar shows the URL `http://localhost:3000/api/books?genre=Classic`. The main interface is divided into several sections:

- Request Section:** The method is `GET` and the URL is `http://localhost:3000/api/books?genre=Classic`. The `Send` button is visible.
- Response Section:** The status is `200 OK` with a response time of `14 ms` and a size of `1.86 KB`. The response body is displayed in JSON format.

The response body is a JSON object with the following structure:

```
{
  "success": true,
  "count": 6,
  "total": 6,
  "totalPages": 1,
  "currentPage": 1,
  "data": [
    {
      "_id": "6a2195a6bd0bb7d6ba5a1eaa",
      "title": "The Great Gatsby",
      "author": "F. Scott Fitzgerald",
      "genre": "Classic",
      "price": 15.99,
      "publishedDate": "1925-04-10T00:00:00.000Z",
      "inStock": true,
      "createdAt": "2026-06-04T15:11:34.397Z",
      "updatedAt": "2026-06-04T15:11:34.397Z",
      "__v": 0
    },
    {
      "_id": "6a219574bd0bb7d6ba5a1ea9",
      "title": "To Kill a Mockingbird",
      "author": "Harper Lee",
      "genre": "Classic",
      "price": 14,
      "publishedDate": "1960-07-11T00:00:00.000Z",
      "inStock": false
    }
  ]
}
```

6. PAGINATION

The screenshot displays the Confluent API client interface. On the left, a sidebar shows a collection named 'My Collection' with a sub-collection 'POST Post data'. The main area shows a REST client request for a GET endpoint: `http://localhost:3000/api/books?page=1&limit=2`. The response is a JSON object with the following structure:

```
1 {
2   "price": 20.00,
3   "inStock": true
4 }
```

The response status is 200 OK, with a response time of 15 ms and a body size of 845 B. The response body is a JSON object with the following structure:

```
1 {
2   "success": true,
3   "count": 2,
4   "total": 7,
5   "totalPages": 4,
6   "currentPage": 1,
7   "data": [
8     {
9       "_id": "6a2195a6bd0bb7d6ba5a1eaa",
10      "title": "The Great Gatsby",
11      "author": "F. Scott Fitzgerald",
12      "genre": "Classic",
13      "price": 15.99,
14      "publishedDate": "1925-04-10T00:00:00.000Z",
15      "inStock": true,
16      "createdAt": "2026-06-04T15:11:34.397Z",
17      "updatedAt": "2026-06-04T15:11:34.397Z",
18      "__v": 0
19    },
20    {
21      "_id": "6a219574bd0bb7d6ba5a1ea9",
22      "title": "To Kill a Mockingbird",
23      "author": "Harper Lee",
24      "genre": "Classic",
25      "price": 14,
26      "publishedDate": "1960-07-11T00:00:00.000Z",
27      "inStock": false
28    }
29  ]
30 }
```